ART 1 — Plan, Prereqs, and Backend (v3-compliant) with Whisper STT + LLM Tool-Calls
0) What we're building (signal flow)

```
User (kiosk mic) ─► Browser
   │  audio blobs
   ▼
Backend
   1) /stt/transcribe  ─► OpenAI Whisper → text            (Speech-to-Text)
:contentReference[oaicite:0]{index=0}
   2) /brain/respond   ─► LLM (function-calling) → tools → text  (Intent + Tools)
:contentReference[oaicite:1]{index=1}
   3) /avatar/speak    ─► HeyGen /v1/streaming.task (talk)     (TTS+video)
:contentReference[oaicite:2]{index=2}
   4) /session/new     ─► HeyGen v3 (LiveKit creds)          (Session)
:contentReference[oaicite:3]{index=3}
```

Browser also renders LiveKit video stream from HeyGen v3 and shows overlays only when intent = "browse/checkout/directions".

Why v3: HeyGen deprecated avatar versions v1 & v2. You must pass version: "v3" and use avatar_id (not avatar_name). Deprecated calls return errors (e.g., 10003, 400014).
HeyGen API Documentation
+2
HeyGen API Documentation
+2

Why custom STT: Built-in voice chat is convenient but doesn't expose transcripts for your external tools. We'll run OpenAI Whisper on the backend to get reliable text, then let an LLM with function-calling decide (show products, add to cart, checkout, directions), and finally speak via HeyGen's streaming.task.
OpenAI Platform
+2
OpenAI Platform
+2

1) Prereqs (do these once)

HeyGen account with an Interactive Avatar + API key. Start with HeyGen quick start.
HeyGen API Documentation

Your avatar_id from HeyGen assets.

Node.js 18+ and npm installed.

OpenAI API key for Whisper STT + LLM. (We'll use Whisper and function calling.)
OpenAI Platform
+2
OpenAI Platform

+2

(Optional) A products DB—you can keep the stub for now.

Security rule: Never expose the HeyGen or OpenAI keys to the browser; keep them in the backend only.

2) Project scaffold (mono-repo)

```
# workspace
mkdir icecream-kiosk && cd icecream-kiosk

# frontend (React + TS + Vite)
npm create vite@latest frontend -- --template react-ts
cd frontend && npm i @heygen/streaming-avatar livekit-client && cd ..

# backend (Node + Express + TS)
mkdir backend && cd backend
npm init -y
npm i express node-fetch multer openai
npm i -D typescript ts-node @types/node @types/express @types/multer
npx tsc --init
```

Create backend/.env (server-only):

```
HEYGEN_API_KEY=YOUR_HEYGEN_API_KEY
HEYGEN_BASE_URL=https://api.heygen.com
OPENAI_API_KEY=YOUR_OPENAI_API_KEY
```

3) Backend — complete v3 server with STT + LLM tools

This file gives you:

/api/session/new — HeyGen v3 session (LiveKit creds)

/api/avatar/speak — proxy to HeyGen streaming.task (talk)

/api/stt/transcribe — OpenAI Whisper STT (accepts audio multipart)

/api/brain/respond — LLM brain with function-calling to stub tools

/api/nlp/interpret, /api/catalog/search, /api/order/checkout, /api/directions/for-pickup — same as before (safe stubs)

References you'll see reflected in code:
• Streaming API + LiveKit integration flow.
HeyGen API Documentation
+1

• streaming.task payload for talking.
HeyGen API Documentation

• Avatar v3 requirement & deprecation.
HeyGen API Documentation
+1

• Whisper speech-to-text endpoint usage.
OpenAI Platform
+1

• OpenAI function calling for tool orchestration.
OpenAI Help Center
+1

• HeyGen's own "add STT to demo project" (same idea—use Whisper to transcribe then speak back).
HeyGen API Documentation

Create backend/src/server.ts:

```
import express from "express";
import fetch from "node-fetch";
import multer from "multer";
import dotenv from "dotenv";
import { OpenAI } from "openai";

dotenv.config();
const app = express();
app.use(express.json());

// ---- config & clients ----
const HEYGEN_API = process.env.HEYGEN_BASE_URL || "https://api.heygen.com";
const HEYGEN_KEY = process.env.HEYGEN_API_KEY;
const OPENAI_KEY = process.env.OPENAI_API_KEY;
if (!HEYGEN_KEY) throw new Error("Missing HEYGEN_API_KEY");
if (!OPENAI_KEY) throw new Error("Missing OPENAI_API_KEY");
const openai = new OpenAI({ apiKey: OPENAI_KEY });

/**
 * 1) Create HeyGen v3 session (HeyGen-managed LiveKit)
 *    v1/v2 are deprecated; pass version:"v3" and avatar_id. (Docs: v1/v2 no longer
supported)
 */
app.post("/api/session/new", async (req, res) => {
  try {
    const { avatarId, language = "en", quality = "high" } = req.body || {};
```

```
    const r = await fetch(`${HEYGEN_API}/v1/streaming.new`, {
      method: "POST",
      headers: { "x-api-key": HEYGEN_KEY!, "Content-Type": "application/json" },
      body: JSON.stringify({
        avatar_id: avatarId,
        version: "v3",       // REQUIRED: v1/v2 deprecated
        language,
        quality,
        voice_mode: false,   // custom STT + tool flow (we speak via streaming.task)
        voice_auto_start: false
      }),
    });
    const data = await r.json();
    if (!r.ok || !data?.data) return res.status(500).json({ error: data });
    const { session_id, livekit_url, access_token } = data.data;
    res.json({ sessionId: session_id, livekitUrl: livekit_url, accessToken: access_token });
  } catch (e:any) {
    res.status(500).json({ error: e?.message || "session.new failed" });
  }
});

/**
 * 2) Speak via avatar (text -> TTS + video)
 *    Proxy so the browser never holds HEYGEN_KEY.
 */
app.post("/api/avatar/speak", async (req, res) => {
  try {
    const { sessionId, text } = req.body || {};
    const r = await fetch(`${HEYGEN_API}/v1/streaming.task`, {
      method: "POST",
      headers: { "x-api-key": HEYGEN_KEY!, "Content-Type": "application/json" },
      body: JSON.stringify({ session_id: sessionId, task_type: "talk", text }),
    });
    const data = await r.json();
    if (!r.ok) return res.status(500).json({ error: data });
    res.json({ ok: true });
  } catch (e:any) {
    res.status(500).json({ error: e?.message || "speak failed" });
  }
});

/**
 * 3) STT — OpenAI Whisper transcription
 *    Accepts audio via multipart/form-data: field "audio" = Blob (webm/wav/mp3).
 *    Returns text transcript.
 */
const upload = multer({ storage: multer.memoryStorage() });
app.post("/api/stt/transcribe", upload.single("audio"), async (req, res) => {
```

```
  try {
    if (!req.file) return res.status(400).json({ error: "missing audio" });
    // Whisper model: "whisper-1" (OpenAI Speech-to-Text guide)
    const transcript = await openai.audio.transcriptions.create({
      file: new File([req.file.buffer], req.file.originalname || "input.webm", { type:
req.file.mimetype }),
      model: "whisper-1"
    });
    res.json({ text: transcript.text });
  } catch (e:any) {
    res.status(500).json({ error: e?.message || "stt failed" });
  }
});

/**
 * 4) LLM Brain — function calling for tools
 *    Tools:
 *      - find_products(query)
 *      - add_to_cart(product_id, qty)
 *      - checkout(cart)
 *      - directions(bin)
 *    This demo uses simple in-memory stubs. Replace with your DB/services.
 */
type CartItem = { id: string; qty: number; price_cents: number };
const PRODUCTS = [
  { id: "p1", name: "Chocolate Cone", price_cents: 12000, image_url: "/img/choc_cone.png",
bin: "FZ-A2-B4" },
  { id: "p2", name: "Vanilla Cup",    price_cents:  9000, image_url: "/img/vanilla_cup.png", bin:
"FZ-B1-A1" },
  { id: "p3", name: "Strawberry Tub", price_cents: 18000, image_url: "/img/straw_tub.png",
bin: "FZ-A2-C2" },
];

function findProducts(query:string){
  const q = query.toLowerCase();
  return PRODUCTS.filter(p => p.name.toLowerCase().includes(q) || (q.includes("chocolate")
&& p.id=="p1"));
}
function addToCart(cart:CartItem[], product_id:string, qty:number){
  const prod = PRODUCTS.find(p=>p.id===product_id) || PRODUCTS[0];
  const idx = cart.findIndex(li=>li.id===prod.id);
  if (idx>=0) cart[idx].qty += qty; else cart.push({ id: prod.id, qty, price_cents: prod.price_cents
});
  return cart;
}
function checkout(cart:CartItem[]){
  const subtotal = cart.reduce((s, li)=> s + li.price_cents*li.qty, 0);
  const tax = Math.round(subtotal * 0.05);
```

```
  const total = subtotal + tax;
  return { subtotal, tax, total };
}
function getDirections(bin:string){
  return { display_name: "Freezer Aisle 2, Bin 4", steps: ["Walk straight 5m","Turn left"], bin,
map_svg_url: "/map/store.svg" };
}

app.post("/api/brain/respond", async (req, res) => {
  try {
    const { utterance, cart=[] } = req.body || {};

    const tools = [
      {
        type: "function",
        function: {
          name: "find_products",
          description: "Search ice-cream catalog by a natural language query.",
          parameters: { type: "object", properties: { query: { type:"string" } }, required: ["query"] }
        }
      },
      {
        type: "function",
        function: {
          name: "add_to_cart",
          description: "Add a product to the cart with quantity.",
          parameters: { type:"object", properties:{ product_id:{type:"string"}, qty:{type:"integer"} },
required:["product_id","qty"] }
        }
      },
      {
        type: "function",
        function: {
          name: "checkout",
          description: "Compute totals for the current cart.",
          parameters: { type:"object", properties:{ cart:{type:"array", items:{type:"object"}}},
required:["cart"] }
        }
      },
      {
        type: "function",
        function: {
          name: "get_directions",
          description: "Get pickup directions for a product bin code.",
          parameters: { type:"object", properties:{ bin:{type:"string"} }, required:["bin"] }
        }
      }
    ];
```

```
    // Use a current chat model that supports function calling.
    const resp = await openai.chat.completions.create({
      model: "gpt-4o-mini", // fast, tool-capable; adjust as needed
      messages: [
        { role: "system", content: "You are a kiosk assistant for an ice-cream shop. Be brief and
helpful." },
        { role: "user", content: utterance }
      ],
      tools
    });

    const choice = resp.choices[0];
    const toolCall = choice?.message?.tool_calls?.[0];
    if (toolCall) {
      const { name, arguments: argsJson } = toolCall.function;
      const args = JSON.parse(argsJson || "{}");

      if (name === "find_products") {
        const items = findProducts(args.query || "");
        // Return an overlay instruction to the client + a short line to speak
        return res.json({ type: "show_products", products: items, response: "Here are some
options I found." });
      }
      if (name === "add_to_cart") {
        const updated = addToCart(cart, args.product_id, args.qty || 1);
        return res.json({ type: "add_to_cart", productId: args.product_id, qty: args.qty || 1, cart:
updated, response: "Added to your cart. Anything else?" });
      }
      if (name === "checkout") {
        const totals = checkout(cart);
        return res.json({ type: "checkout", receipt: totals, response: `Your total is
₹${(totals.total/100).toFixed(2)}. Would you like anything else?` });
      }
      if (name === "get_directions") {
        const info = getDirections(args.bin || "FZ-A2-B4");
        return res.json({ type: "directions", directions: info, response: `Please head to
${info.display_name}.` });
      }
    }

    // No tool call → small talk fallback
    return res.json({ type: "chat", response: choice.message.content || "How can I help you
choose ice-cream?" });
  } catch (e:any) {
    res.status(500).json({ error: e?.message || "brain failed" });
  }
});
```

```
/**
 * 5) (Same safe stubs as before) Direct endpoints you may keep if the frontend calls them directly.
 */
app.post("/api/nlp/interpret", (req, res) => {
  // Keep for backwards-compat, but /api/brain/respond supersedes it.
  res.json({ type: "chat", response: "Use /api/brain/respond for intent + tools." });
});

app.get("/api/catalog/search", (req, res) => {
  const q = (req.query.q as string || "").toLowerCase();
  res.json({ results: PRODUCTS.filter(p => p.name.toLowerCase().includes(q)) });
});

app.post("/api/order/checkout", (req, res) => {
  const cart: CartItem[] = req.body?.cart || [];
  const totals = checkout(cart);
  res.json({ receipt: totals, receipt_url: "/receipt/123" });
});

app.get("/api/directions/for-pickup", (req, res) => {
  const bin = (req.query.bin as string) || "FZ-A2-B4";
  res.json(getDirections(bin));
});

const PORT = process.env.PORT || 8080;
app.listen(PORT, () => console.log(`Backend listening on ${PORT}`));
```

Why this is correct (and current):

HeyGen v3 session is mandatory; v1/v2 are deprecated (returns errors).
HeyGen API Documentation
+1

LiveKit + Streaming API is the endorsed real-time path.
HeyGen API Documentation
+1

/v1/streaming.task is the supported way to make the avatar speak text you provide.
HeyGen API Documentation

OpenAI Whisper (whisper-1) is the documented Speech-to-Text API.
OpenAI Platform
+1

Function calling is the official mechanism to route LLM to tools (find products, add to cart, etc.).
OpenAI Help Center
+1

HeyGen's own guide demonstrates the same "Whisper STT → avatar speaks" idea for their demo.
HeyGen API Documentation

4) Test the backend quickly

From /backend:

```
npx ts-node src/server.ts
```

Quick checks (you can use curl or Postman):

```
# Create a v3 session (replace YOUR_AVATAR_ID)
curl -sX POST http://localhost:8080/api/session/new \
  -H "Content-Type: application/json" \
  -d '{"avatarId":"YOUR_AVATAR_ID","language":"en"}' | jq

# Make the avatar speak (after session exists; paste the real sessionId)
curl -sX POST http://localhost:8080/api/avatar/speak \
  -H "Content-Type: application/json" \
  -d '{"sessionId":"<SESSION_ID>","text":"Hello from the kiosk!"}' | jq
```

For STT you'll post an audio blob from the browser; wiring that is in PART 2. The backend will return { text: "transcript..." }.

5) Voice + Brain – end-to-end turn (how the loop runs)

Browser records a short audio clip (user finished speaking) and POSTs to /api/stt/transcribe.

Backend uses Whisper (whisper-1) to return text.
OpenAI Platform
+1

Browser POSTs text (plus a tiny cart context) to /api/brain/respond.

LLM uses function calling to pick a tool: find_products, add_to_cart, checkout, or get_directions.
OpenAI Help Center
+1

Backend executes the tool and returns a JSON result (and a short response to speak).

Browser calls /api/avatar/speak, which forwards to HeyGen /v1/streaming.task → the avatar speaks and video plays via LiveKit.
HeyGen API Documentation

If type === "show_products" or "directions", the overlay appears with images or map (PART 2).

6) Notes on accuracy, reliability & firewalls

STT isn't magic; Whisper is strong but can mishear in noise. Consider adding a push-to-talk button and a short VAD timeout. (Media and browser handling finished in PART 2). General caveats in the press exist; test in your kiosk's acoustic environment.
AP News

WebRTC sometimes collides with corporate firewalls; if you deploy in enterprise, follow HeyGen's firewall guidance (ports, TURN).
HeyGen API Documentation

Keep sessions short; tear down on idle (session best practices).

PART 2 — Frontend (React + TypeScript) with Whisper STT & Overlay UX

0) Recap of what Part 1 gives you

From Part 1 you already have:

- A backend endpoint **/api/session/new** that returns `{ sessionId, livekitUrl, accessToken }` for a HeyGen v3 avatar session.
- A backend endpoint **/api/avatar/speak** to send text to the avatar.
- A backend endpoint **/api/stt/transcribe** that accepts audio and returns a transcript via OpenAI Whisper.
- A backend endpoint **/api/brain/respond** that accepts `{ utterance, cart }` and returns structured JSON `{ type: "...", ... , response: "..." }` indicating what overlay to show, what to say, etc.

- Overlay types: `show_products`, `add_to_cart`, `checkout`, `directions`, and possibly `chat`.
- Session requirement: avatar version “v3” with `avatar_id`.
- UX rule: No product panels at start; only after intent triggers the overlay.

---

1) Project Setup in Frontend

In your `frontend` directory (from Part 1):

```
cd frontend
npm install @heygen/streaming-avatar livekit-client
```

You already created a Vite React-TS project earlier; now we layer in components, services, and logic.

---

2) Folder & File Structure

```
frontend/
 ├ src/
 |    ├ components/
 |    |    Hero.tsx
 |    |    AvatarStream.tsx
 |    |    OverlayManager.tsx
 |    |    ProductOverlay.tsx
 |    |    DirectionsOverlay.tsx
 |    |    MicBar.tsx
 |    ├ lib/
 |    |    api.ts
 |    |    stt.ts
 |    ├ pages/
 |    |    Kiosk.tsx
 |    ├ App.tsx
 |    └ main.tsx
```

We’ll build each file in order.

---

3) `lib/api.ts` – API service for the backend

Create `frontend/src/lib/api.ts`:

```
// frontend/src/lib/api.ts

export async function newSession(avatarId: string) {
  const r = await fetch("/api/session/new", {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify({ avatarId, language: "en", quality: "high"
}),
  });
  if (!r.ok) throw new Error("session.new failed");
  return (await r.json()) as { sessionId: string; livekitUrl:
string; accessToken: string };
}

export async function speak(sessionId: string, text: string) {
  const r = await fetch("/api/avatar/speak", {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify({ sessionId, text }),
  });
  if (!r.ok) throw new Error("speak failed");
}

export async function transcribeAudio(audioBlob: Blob) {
  const form = new FormData();
  form.append("audio", audioBlob, "input.webm");
  const r = await fetch("/api/stt/transcribe", {
    method: "POST",
    body: form,
  });
  if (!r.ok) throw new Error("stt failed");
  const j = await r.json();
  return j.text as string;
}

export async function brainRespond(utterance: string, cart: any[]) {
```

```
  const r = await fetch("/api/brain/respond", {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify({ utterance, cart }),
  });
  if (!r.ok) throw new Error("brain failed");
  return r.json() as Promise<any>;
}
```

This gives you typed calls to your backend for session, speak, STT, and brain responses.

---

4) `lib/stt.ts` – Microphone capture for Whisper STT

Create `frontend/src/lib/stt.ts`:

```
// frontend/src/lib/stt.ts

export async function captureAudio(seconds = 3): Promise<Blob> {
  const stream = await navigator.mediaDevices.getUserMedia({ audio:
true });
  const recorder = new MediaRecorder(stream, { mimeType:
"audio/webm" });
  const chunks: BlobPart[] = [];

  recorder.ondataavailable = (e) => {
    if (e.data.size > 0) {
      chunks.push(e.data);
    }
  };

  recorder.start();
  await new Promise((r) => setTimeout(r, seconds * 1000));
  recorder.stop();

  await new Promise((r) => {
    recorder.onstop = () => r(undefined);
  });

  stream.getTracks().forEach((t) => t.stop());
```

```
  return new Blob(chunks, { type: "audio/webm" });
}
```

Explanation:

- We request mic access (`getUserMedia`).
- Record for a set time (default 3 seconds).
- Return a Blob suitable for sending to `/api/stt/transcribe`.
- For kiosk use you may replace this with push-to-talk UI (not automatic continuous recording) to reduce errors.

---

5) UI Components

`Hero.tsx` – Landing (start) screen

Create `frontend/src/components/Hero.tsx`:

```
// frontend/src/components/Hero.tsx

import React from "react";

type HeroProps = { onStart: () => void };

export function Hero({ onStart }: HeroProps) {
  return (
    <section
      className="w-screen h-screen flex items-center justify-center
bg-cover bg-center"
      style={{ backgroundImage: "url('/assets/hero-icecream.jpg')"
}}
    >
      <div className="text-center max-w-xl">
        <h1 className="text-5xl font-extrabold text-white
drop-shadow-lg">
          Welcome to Scoop Haven 🍦
        </h1>
```

```
        <p className="mt-3 text-lg text-white/90">
          Tap Start to talk with our ice-cream assistant.
        </p>
        <button
          onClick={onStart}
          className="mt-8 px-8 py-4 rounded-2xl bg-white
text-purple-700 font-semibold hover:bg-purple-100"
        >
          Start
        </button>
      </div>
    </section>
  );
}
```

`AvatarStream.tsx` – Connect to LiveKit and HeyGen avatar

Create `frontend/src/components/AvatarStream.tsx`:

```
// frontend/src/components/AvatarStream.tsx

import React, { useEffect, useRef } from "react";
import { connect, Room } from "livekit-client";
import StreamingAvatar from "@heygen/streaming-avatar";

type AvatarStreamProps = {
  session: { sessionId: string; livekitUrl: string; accessToken:
string };
  avatarId: string;
  onReady: (deps: { room: Room; avatar: StreamingAvatar }) => void;
};

export function AvatarStream({ session, avatarId, onReady }:
AvatarStreamProps) {
  const videoRef = useRef<HTMLVideoElement>(null);

  useEffect(() => {
    let room: Room | null = null;
    let avatar: StreamingAvatar | null = null;

    (async () => {
```

```
      // Connect LiveKit room
      room = await connect(session.livekitUrl, session.accessToken);
      room.on("trackSubscribed", (track) => {
        if (track.kind === "video" && videoRef.current) {
          videoRef.current.srcObject = new
MediaStream([track.mediaStreamTrack]);
        }
      });

      // Create HeyGen avatar
      avatar = new StreamingAvatar({ token: session.accessToken });
      await avatar.createStartAvatar({
        avatarId,
        version: "v3",
        language: "en",
        quality: "high",
      });

      onReady({ room, avatar });
    })();

    return () => {
      try {
        avatar?.stop?.();
      } catch {}
      try {
        room?.disconnect?.();
      } catch {}
      if (videoRef.current) videoRef.current.srcObject = null;
    };
  }, [session, avatarId, onReady]);

  return (
    <video
      ref={videoRef}
      autoPlay
      muted
      playsInline
      className="w-full max-w-3xl rounded-xl shadow-lg"
    />
  );
```

```
}
```

`OverlayManager.tsx`, `ProductOverlay.tsx`, `DirectionsOverlay.tsx`

These were defined in Part 1 (you'll reuse those exact components). They manage the overlay logic (showing products or directions) when triggered.

`MicBar.tsx` – UI for push-to-talk or text input

Create `frontend/src/components/MicBar.tsx`:

```
// frontend/src/components/MicBar.tsx

import React, { useState } from "react";
import { captureAudio } from "../lib/stt";

type MicBarProps = { onUtterance: (text: string) => void };

export function MicBar({ onUtterance }: MicBarProps) {
  const [busy, setBusy] = useState(false);

  const handleSpeak = async () => {
    try {
      setBusy(true);
      const blob = await captureAudio(3); // record 3 seconds
      const transcript = await fetch("/api/stt/transcribe", {
        method: "POST",
        body: blob,
      }).then((r) => r.json()).then((j) => j.text);
      if (transcript) onUtterance(transcript);
    } catch (e) {
      console.error("STT error", e);
      alert("Audio capture failed. Please try again.");
    } finally {
      setBusy(false);
    }
  };

  return (
    <div className="flex gap-2 items-center">
```

```
      <button
        onClick={handleSpeak}
        disabled={busy}
        className="px-6 py-3 rounded-xl bg-purple-600 text-white
hover:bg-purple-500 disabled:opacity-50"
      >
        {busy ? "Listening…" : "Hold to Talk"}
      </button>
      <input
        type="text"
        placeholder="Or type here"
        className="flex-1 px-3 py-2 rounded-xl border"
        onKeyDown={(e) => {
          if (e.key === "Enter") onUtterance((e.target as
HTMLInputElement).value);
        }}
      />
    </div>
  );
}
```

Note: This sends recorded audio blob to `/api/stt/transcribe`. You could integrate with `transcribeAudio` helper defined earlier.

---

6) `pages/Kiosk.tsx` – Main state machine tying everything together

Create `frontend/src/pages/Kiosk.tsx`:

```
// frontend/src/pages/Kiosk.tsx

import React, { useCallback, useEffect, useState } from "react";
import { Hero } from "../components/Hero";
import { AvatarStream } from "../components/AvatarStream";
import { OverlayManager, Overlay } from
"../components/OverlayManager";
import { MicBar } from "../components/MicBar";
import { newSession, speak, brainRespond } from "../lib/api";
```

```
export function Kiosk() {
  const [started, setStarted] = useState(false);
  const [session, setSession] = useState<{ sessionId:string;
livekitUrl:string; accessToken:string } | null>(null);
  const [overlay, setOverlay] = useState<Overlay>(null);
  const [cart, setCart] = useState<{ id:string; qty:number;
price_cents:number }[]>([]);
  const avatarId = "YOUR_AVATAR_ID"; // ← Replace with your actual
avatar ID

  // Event listeners for add-to-cart & closing overlays
  useEffect(() => {
    const onAdd = (e:any) => {
      const id = e.detail.id as string;
      const price = id === "p1" ? 12000 : id === "p2" ? 9000 :
18000;
      setCart((c) => {
        const idx = c.findIndex(li => li.id === id);
        if (idx >= 0) {
          const copy = [...c];
          copy[idx].qty += 1;
          return copy;
        }
        return [...c, { id, qty:1, price_cents:price }];
      });
    };
    const onClose = () => setOverlay(null);

    window.addEventListener("add-to-cart", onAdd);
    window.addEventListener("close-overlay", onClose);
    return () => {
      window.removeEventListener("add-to-cart", onAdd);
      window.removeEventListener("close-overlay", onClose);
    };
  }, []);

  const handleUtterance = useCallback(async (text:string) => {
    if (!session) return;
    try {
      const intent = await brainRespond(text, cart);
      // Speak first
```

```
      await speak(session.sessionId, intent.response || "Okay!");
      // Then show overlay if needed
      if (intent.type === "show_products") {
        setOverlay({ type:"products", data: intent.products });
      } else if (intent.type === "directions") {
        setOverlay({ type:"directions", data: intent.directions });
      } else if (intent.type === "add_to_cart") {
        setCart(intent.cart);
        setOverlay(null); // Hide products overlay after add
      } else if (intent.type === "checkout") {
        // you could navigate to a ReceiptOverlay, for now just hide
overlay
        setOverlay(null);
      } else {
        // chat fallback
        setOverlay(null);
      }
    } catch (e) {
      console.error("brain error", e);
      await speak(session.sessionId, "I'm sorry — something went
wrong. Please try again.");
    }
  }, [session, cart]);

  const startSession = useCallback(async () => {
    try {
      const s = await newSession(avatarId);
      setSession(s);
      setStarted(true);
      // greet
      setTimeout(() => {
        speak(s.sessionId, "Hi! Welcome to Scoop Haven. How can I
help you today?");
      }, 500);
    } catch (e) {
      console.error("session start error", e);
      alert("Session failed. Please restart.");
    }
  }, []);

  return (
```

```
    <div className="relative w-screen h-screen bg-black text-white
overflow-hidden">
      {!started ? (
        <Hero onStart={startSession} />
      ) : session ? (
        <>
          <AvatarStream session={session} avatarId={avatarId}
onReady={() => {}} />
          <OverlayManager overlay={overlay} />
          <div className="absolute bottom-6 left-1/2
-translate-x-1/2 w-3/4 max-w-3xl">
            <MicBar onUtterance={handleUtterance} />
          </div>
        </>
      ) : (
        <div className="flex items-center justify-center h-full">
          <p>Loading…</p>
        </div>
      )}
    </div>
  );
}
```

App.tsx

```
// frontend/src/App.tsx

import React from "react";
import { Kiosk } from "./pages/Kiosk";

export default function App() {
  return <Kiosk />;
}
```

---

7) Running & Testing the Frontend

From your `frontend` folder:

```
npm run dev
```

Then open the displayed URL (often `http://localhost:5173`) in a browser. Because of mic use and WebRTC, you *must* be on **HTTPS** for production; for local dev you may use `localhost`.

Test steps:

1. Landing screen: shows hero section and Start button only.
2. Click **Start**: session created, avatar appears (video) and initial greeting is spoken.
3. In the MicBar: either
    - Hold the **"Hold to Talk"** button to record 3 seconds, or
    - Type into the text box and press Enter.
4. Say or type "show chocolate cone" → You should see the product overlay appear with cards (images, price).
5. Click **Add** on a card → an `add-to-cart` event triggers; cart is updated silently; the avatar will speak "Added to your cart…"
6. Type "checkout" → the overlay hides; avatar says "Your total is ₹…"
7. Type "where is it?" → directions overlay appears: "Freezer Aisle 2, Bin 4" with map and steps.
8. Idle for ~60–90 seconds → you should implement a timer (later) that resets to the hero screen.

---

8) Integrating Overlay Transitions & Style Enhancements

To make the experience smooth:

- Use `className="fixed inset-0 bg-black/70 backdrop-blur-sm z-50"` for overlays so they fade in and blur the background.
- Use CSS transitions or Framer Motion for fade/slide animations.
- Ensure the video stays visible behind overlays (so avatar remains visible while overlay shows).

- Use large fonts and touch-friendly button sizes (for kiosk use).
- Preload product images (`<link rel="preload">`) so overlay appears fast.

---

9) Going Live Considerations

- For deployment, serve frontend over **HTTPS** (mic + camera require secure origin).
- Ensure CORS and proxy settings: your frontend `fetch("/api/...")` goes to your backend (configure Vite proxy or same host).
- Use `navigator.mediaDevices.getUserMedia({ audio: true })` only when user clicks Start (browser requires user gesture).
- Test on actual kiosk hardware (touch screen, ambient noise).
- Implement idle timeout: e.g., after **90 seconds** of no utterance, call backend to end session and return to hero screen. Use `setTimeout` and reset timer on any user interaction.

✅ You've now completed PART 2

You have a fully wired **frontend React + TS** system that:

- Creates and connects to HeyGen v3 avatar via LiveKit.
- Captures user audio (or typed input), sends to Whisper STT → brain → overlay/UI update → avatar speaks.
- Uses **overlay UX** (not full panels) so the UI remains minimal until needed.
- Maps to the backend logic built in Part 1.

🧠 Revised Architecture (with Make.com + Airtable integration)

```
[ Kiosk Frontend ]
   |
```

```
   ├─ /api/stt/transcribe     → Whisper STT (speech → text)
   ├─ /api/brain/respond      → LLM + tool-calls (decides what to
do)
   │      └─ tool: find_products  → calls → /api/make/products
   │      └─ tool: add_to_cart    → client only
   │      └─ tool: checkout       → calls → /api/make/order
   │      └─ tool: directions     → static /api/make/directions
   │
   └─ /api/avatar/speak        → HeyGen streaming.task (avatar
speaks)
   │
[ Backend ]
   ├─ /api/make/products   → forwards query → Make.com "Find
Products" webhook
   ├─ /api/make/order      → forwards checkout payload → Make.com
"Create Order" webhook
   │                         Make.com scenario stores to Airtable &
returns JSON
   └─ /api/make/directions → optional static JSON or Map API call
   │
[ Make.com Scenarios ]
   ├─ Webhook: Find Products → Airtable "Products" base → returns
list (name, price, image_url, bin)
   ├─ Webhook: Create Order  → Airtable "Orders" base → returns
confirmation, total, etc.
   └─ (future) Webhook: Directions → optional Airtable "Bins" lookup
```

Flow summary

1. Avatar greets user.
2. User speaks: audio → Whisper STT.
3. LLM brain interprets ("show chocolate").
4. Tool call → `/api/make/products`.
5. Backend → forwards to Make.com webhook (fetch).
6. Make → Airtable → returns JSON (products with images).
7. Frontend overlay displays those images.

8. User selects item → Add to cart → Checkout → `/api/make/order` webhook.
9. Make creates record in Airtable “Orders” and returns receipt + total.
10. Avatar speaks confirmation and directions.

🧱 Implementation Details

1. Configure Make.com

You’ll have **two scenarios** in Make:

a. “Find Products” Scenario

- Trigger: **Custom Webhook** (`hook_find_products`).
- Action 1: Airtable → Search Records in “Products” base.

Return bundle:

```
{
  "products": [
    {
      "id": "recC123",
      "name": "Chocolate Cone",
      "price_cents": 12000,
      "image_url":
"https://dl.airtable.com/.attachments/…/cone.jpg",
      "bin": "FZ-A2-B4"
    }
  ]
}
```

-

b. “Create Order” Scenario

- Trigger: **Custom Webhook** (`hook_create_order`).
- Action 1: Airtable → Create Record in “Orders”.

Return bundle:

```
{
  "order_id": "rec0456",
  "receipt": { "subtotal": 24000, "tax": 1200, "total": 25200 },
  "status": "ok"
}
```

-

Take note of each **webhook URL** Make gives you (for example: `https://hook.make.com/abCDef123Find` and `https://hook.make.com/xyZ987Order`).

---

2. Backend updates (connects to Make webhooks)

In your **backend/src/server.ts** (from Part 1), add after imports:

```
const MAKE_PRODUCTS_HOOK = process.env.MAKE_PRODUCTS_HOOK; // your
make webhook url
const MAKE_ORDER_HOOK = process.env.MAKE_ORDER_HOOK;
```

Add these routes **before** `app.listen`:

```
/**
 * 6) Forward to Make.com "Find Products" scenario.
 *    Called by brain.respond()'s find_products tool.
 */
app.post("/api/make/products", async (req, res) => {
  try {
    const { query } = req.body || {};
    const r = await fetch(MAKE_PRODUCTS_HOOK!, {
      method: "POST",
      headers: { "Content-Type": "application/json" },
      body: JSON.stringify({ query }),
    });
    const data = await r.json();
    res.json(data);
  } catch (e:any) {
    res.status(500).json({ error: e.message || "make products
failed" });
```

```
  }
});

/**
 * 7) Forward to Make.com "Create Order" scenario.
 *    Called during checkout.
 */
app.post("/api/make/order", async (req, res) => {
  try {
    const { cart, total } = req.body || {};
    const r = await fetch(MAKE_ORDER_HOOK!, {
      method: "POST",
      headers: { "Content-Type": "application/json" },
      body: JSON.stringify({ cart, total }),
    });
    const data = await r.json();
    res.json(data);
  } catch (e:any) {
    res.status(500).json({ error: e.message || "make order failed"
});
  }
});
```

3. Update the “brain” function calls

Inside the `/api/brain/respond` endpoint from Part 1, **replace** the local `findProducts()` and `checkout()` with Make webhook calls:

```
// replace findProducts()
async function findProductsViaMake(query:string) {
  const r = await fetch(`${process.env.LOCAL_URL ||
"http://localhost:8080"}/api/make/products`, {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify({ query })
  });
  const data = await r.json();
  return data.products || [];
}

// replace checkout()
```

```
async function checkoutViaMake(cart:CartItem[]) {
  const r = await fetch(`${process.env.LOCAL_URL ||
"http://localhost:8080"}/api/make/order`, {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify({ cart, total: cart.reduce((s,
c)=>s+c.price_cents*c.qty, 0) })
  });
  const data = await r.json();
  return data.receipt || { subtotal:0, tax:0, total:0 };
}
```

Then, in the tool-call logic, call these async helpers:

```
if (name === "find_products") {
  const items = await findProductsViaMake(args.query || "");
  return res.json({ type:"show_products", products: items,
response:"Here are some options I found." });
}

if (name === "checkout") {
  const totals = await checkoutViaMake(cart);
  return res.json({ type:"checkout", receipt: totals, response:`Your
total is ₹${(totals.total/100).toFixed(2)}. Would you like anything
else?` });
}
```

All other tools stay the same.

---

4. Handling images and missing data

- Make sure the Airtable “Products” table has an **Attachments** field; it returns `image[0].url` in the webhook payload.

In the frontend overlay, always check `p.image_url` and display a fallback if missing:

```
<img src={p.image_url || "/assets/placeholder.png"} … />
```

-

- For rich info (description, flavors, etc.), include extra fields in Airtable and pass them through Make.

---

5. Error handling & fallback UX

Inside `/api/make/products` and `/api/make/order` we already catch errors.
On the frontend, if `brainRespond()` returns `{ error: … }`, make the avatar say:

```
await speak(session.sessionId, "I'm having trouble accessing our menu. Please try again in a moment.");
```

---

6. Benefits of this MVP structure

- **Zero direct DB** — all state (products/orders) lives in Airtable.
- **Make.com** acts as your orchestrator (you can visually modify flows).
- **Webhooks** decouple your backend; no need to redeploy for DB schema tweaks.
- **Frontend** stays unchanged; it just renders what comes from Make.
- **Future-proof** — easily switch to real Postgres or Neon later by replacing `/api/make/*` with DB calls.

---

7. Testing

1. Start backend (`npx ts-node src/server.ts`).
2. Start frontend (`npm run dev`).
3. Trigger “show chocolate cone” → watch Make’s “Find Products” webhook run.
4. Confirm Airtable returns product JSON → overlay displays product cards with real images.
5. Add to cart → checkout → watch Make “Create Order” scenario run and log the order in Airtable.

6. Confirm avatar reads total aloud and overlay hides.

✅ Outcome

Your kiosk now runs fully end-to-end:

- Whisper → text
- LLM brain (function-calling) → tool triggers
- Tool → Make.com webhook → Airtable data
- HeyGen avatar → speaks + video
- React frontend → shows overlay dynamically.

This keeps your MVP lean, **no custom DB**, and every part can later scale up to Neon/Postgres without breaking UI or agent logic.

Fantastic — welcome to **Part 3**, the final stretch of your Ice-Cream Kiosk MVP.
You already have:

- Part 1 → Backend with HeyGen v3 + OpenAI Whisper + LLM brain + Make/Airtable webhooks.
- Part 2 → React + TypeScript frontend with avatar stream, overlays, and audio capture.

Now we'll cover **deployment, environment setup, testing, and hardening** — everything you need to run this on a live kiosk or staging server.

🏁 PART 3 — Deployment & Testing Guide

1️⃣ Folder Structure Recap

Your root should now look like this:

```
icecream-kiosk/
 ├ frontend/
 │    ├ src/...
 │    ├ package.json
 │    ├ vite.config.ts
 ├ backend/
 │    ├ src/server.ts
 │    ├ package.json
 │    ├ .env
 ├ docker-compose.yml      ← (we'll create here)
 └ README.md
```

2️⃣ Environment Variables

In `backend/.env`:

```
# HeyGen
HEYGEN_API_KEY=YOUR_HEYGEN_KEY
HEYGEN_BASE_URL=https://api.heygen.com

# OpenAI
OPENAI_API_KEY=YOUR_OPENAI_KEY

# Make.com Webhooks
MAKE_PRODUCTS_HOOK=https://hook.make.com/abc123_products
MAKE_ORDER_HOOK=https://hook.make.com/xyz987_order

# Optional base URL (used for self-calls)
LOCAL_URL=http://backend:8080
PORT=8080
```

In `frontend/.env` (for build-time only):

```
VITE_API_BASE=/api
VITE_HEYGEN_ASSET_PATH=/assets
```

> Never store API keys in the frontend.

3️⃣ Proxy Configuration (Vite → Backend)

In `frontend/vite.config.ts`:

```
import { defineConfig } from 'vite';
import react from '@vitejs/plugin-react';

export default defineConfig({
  plugins: [react()],
  server: {
    proxy: {
      '/api': 'http://localhost:8080', // proxy backend during dev
    },
  },
});
```

4️⃣ Docker Setup (Optional but Recommended)

Create a root-level `docker-compose.yml`:

```
version: "3.9"
services:
  backend:
    build: ./backend
    ports:
      - "8080:8080"
    env_file:
      - ./backend/.env
    command: ["npx", "ts-node", "src/server.ts"]
    volumes:
      - ./backend:/app
    working_dir: /app

  frontend:
    build: ./frontend
    ports:
      - "5173:5173"
    command: ["npm", "run", "dev"]
    volumes:
      - ./frontend:/app
```

```
    working_dir: /app
```

For production, you'll later use a static build (see section 7).

5️⃣ HTTPS Requirement

Browsers block mic/WebRTC on plain HTTP.
For development use:

```
npm install --save-dev vite-plugin-mkcert
```

Then in `vite.config.ts`:

```
import mkcert from 'vite-plugin-mkcert';

export default defineConfig({
  plugins: [react(), mkcert()],
  server: {
    https: true,
    proxy: { '/api': 'http://localhost:8080' },
  },
});
```

This gives you trusted HTTPS on `https://localhost:5173`.

6️⃣ Testing Checklist (Local Loop)

Step 1 – Run Everything

```
# Terminal A
cd backend
npx ts-node src/server.ts

# Terminal B
cd frontend
npm run dev
```

Step 2 – Open in browser

Visit `https://localhost:5173`.

Step 3 – Test flow

1. Click **Start** → avatar video appears, speaks greeting.
2. Press **Hold to Talk** or type “show chocolate cone”.
3. Verify:
    - Audio recorded → `/api/stt/transcribe` returns text.
    - LLM brain → calls `/api/make/products` webhook.
    - Make scenario → Airtable returns JSON.
    - Overlay displays product cards with real images.
    - Avatar says “Here are some options I found.”
4. Click **Add to Cart** → avatar confirms.
5. Type “checkout” → Make “Create Order” webhook fires, Airtable logs order.
6. Avatar says total, directions overlay appears.
7. Idle 60 s → app resets to Hero screen (implement timer below).

7️⃣ Production Build & Serve

Build frontend

```
cd frontend
npm run build
```

Vite outputs to `dist/`.

Serve static frontend from backend

Install `serve-static`:

```
cd backend
npm install serve-static
```

In `backend/src/server.ts` add at top:

```
import path from "path";
import serveStatic from "serve-static";
```

Below all API routes, before `app.listen`:

```
// Serve built frontend (after `npm run build`)
app.use(serveStatic(path.join(__dirname, "../../frontend/dist")));
app.get("*", (req, res) =>
  res.sendFile(path.join(__dirname,
"../../frontend/dist/index.html"))
);
```

Then run:

```
npm run build --prefix frontend
npx ts-node src/server.ts
```

Everything is now on one port (8080 → your backend + frontend).

Deploy this single container to Render, Railway, Fly.io, or VPS.

---

8 Idle Timer (Reset to Hero)

In `Kiosk.tsx`, add:

```
useEffect(() => {
  if (!started) return;
  const timer = setTimeout(() => window.location.reload(), 90_000);
  const reset = () => { clearTimeout(timer); setTimeout(() =>
window.location.reload(), 90_000); };
  window.addEventListener("click", reset);
  window.addEventListener("keydown", reset);
  return () => {
    clearTimeout(timer);
```

```
    window.removeEventListener("click", reset);
    window.removeEventListener("keydown", reset);
  };
}, [started]);
```

9️⃣ Logging & Monitoring

In `server.ts`, log every incoming webhook and tool call:

```
app.use((req, _res, next) => {
  console.log(`[${new Date().toISOString()}] ${req.method}
${req.path}`);
  next();
});
```

-
- Log `Whisper` transcripts and LLM responses for debugging.
- In Make.com, enable “Save data of each execution”.

🔟 Security & Performance Tips

Topic	Recommendation
HTTPS	Mandatory for mic & WebRTC.
Rate limit	Throttle `/api/stt/transcribe` to 1 req/sec per client.
Compression	Use `compression` npm for Express responses.
Cache	Cache product data from Make for ~30 s to reduce webhook cost.
LLM safety	Trim Whisper text to <= 200 chars before sending to LLM.
Avatar sessions	Close session (`/api/session/end`) when reloading.
Error overlays	On error, show “Network Issue – Please Try Again”.

🔧 Optional Enhancements

Feature	How
Analytics	Add Mixpanel or PostHog events on “Start Session”, “Add to Cart”.
Multi-language	Pass `language` from Whisper result to avatar.
Receipt Overlay	Add a third overlay type `receipt` displaying Make response.
Kiosk Mode	Use Chrome command: `chrome --kiosk https://yourdomain`.
Hardware Button	Attach GPIO press → trigger Start button via JavaScript event.

✅ Final Verification Before Demo

Check	Pass
🧠 HeyGen avatar (v3) connects OK	✅
🎤 Whisper returns clean transcripts	✅
🧩 Make.com hooks respond with products & orders	✅
🖼️ Images render correctly in overlay	✅
🗣️ Avatar speaks back each response	✅
📄 Order record created in Airtable	✅
💳 Total shown and spoken in ₹	✅
🧭 Directions overlay shows map/steps	✅
⏱️ Idle timeout resets to Hero	✅

🚀 You’re Now Production-Ready

- **Frontend:** React + TS (Vite, HTTPS)
- **Backend:** Node + Express with HeyGen v3 + Whisper + LLM brain + Make webhooks

- **Database:** Airtable (via Make.com)
- **Voice/Video:** HeyGen managed LiveKit
- **UX:** Dynamic overlays, intent-driven conversation

This architecture is clean, modular, and MVP-ready for real-world kiosks.